

SmolVLM

Redefining Small and Efficient Multimodal Models

Hugging Face & Stanford University

arXiv: 2504.05299 | April 2025

Audience: ML Researchers · On-Device AI Engineers · Graduate Students

Sub-1GB inference · 3 model sizes (256M / 500M / 2.2B) · Outperforms Ldefics-80B · Fully open-source

Large VLMs Are Powerful but Impractical for Edge Deployment

The gap between capability and deployability

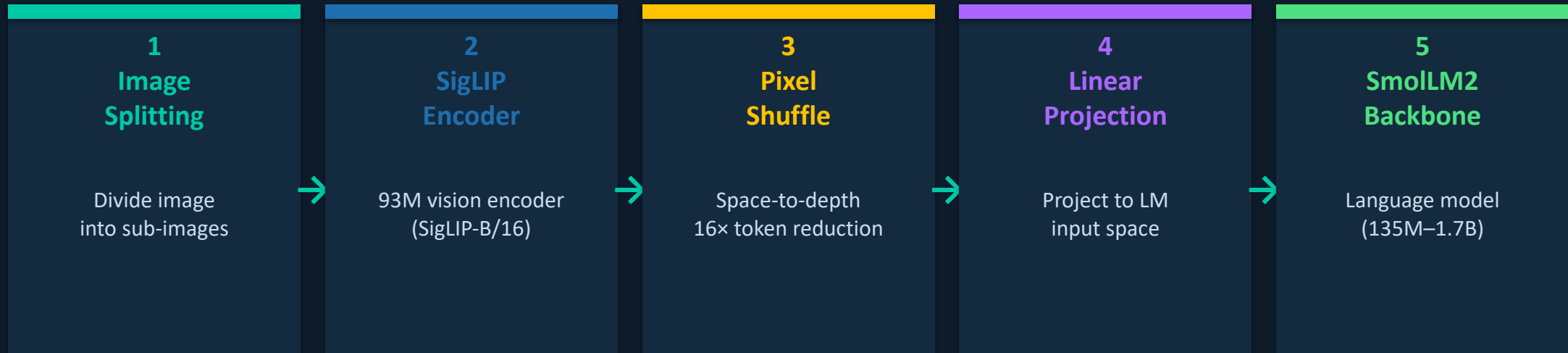
- State-of-the-art VLMs require 20–80 GB GPU RAM
- Cloud inference introduces latency & privacy risks
- Mobile devices have 4–16 GB total RAM
- Existing small VLMs sacrifice too much accuracy
- No viable path to real-time on-device multimodal AI

Model Size vs. RAM Requirement		
Model	Params	RAM (batch=1)
Idefics-80B	80B	~160 GB
Molmo-A1B-7B	7B	27.7 GB
InternVL2-2B	2B	~8 GB
SmoIVLM-2.2B	2.2B	4.9 GB
SmoIVLM-500M	500M	1.2 GB
SmoIVLM-256M ✓	256M	0.8 GB

SmoIVLM-256M: < 1 GB RAM — enabling true on-device multimodal AI

SmolVLM Architecture: Image Splitting, Pixel Shuffle & SmolLM2

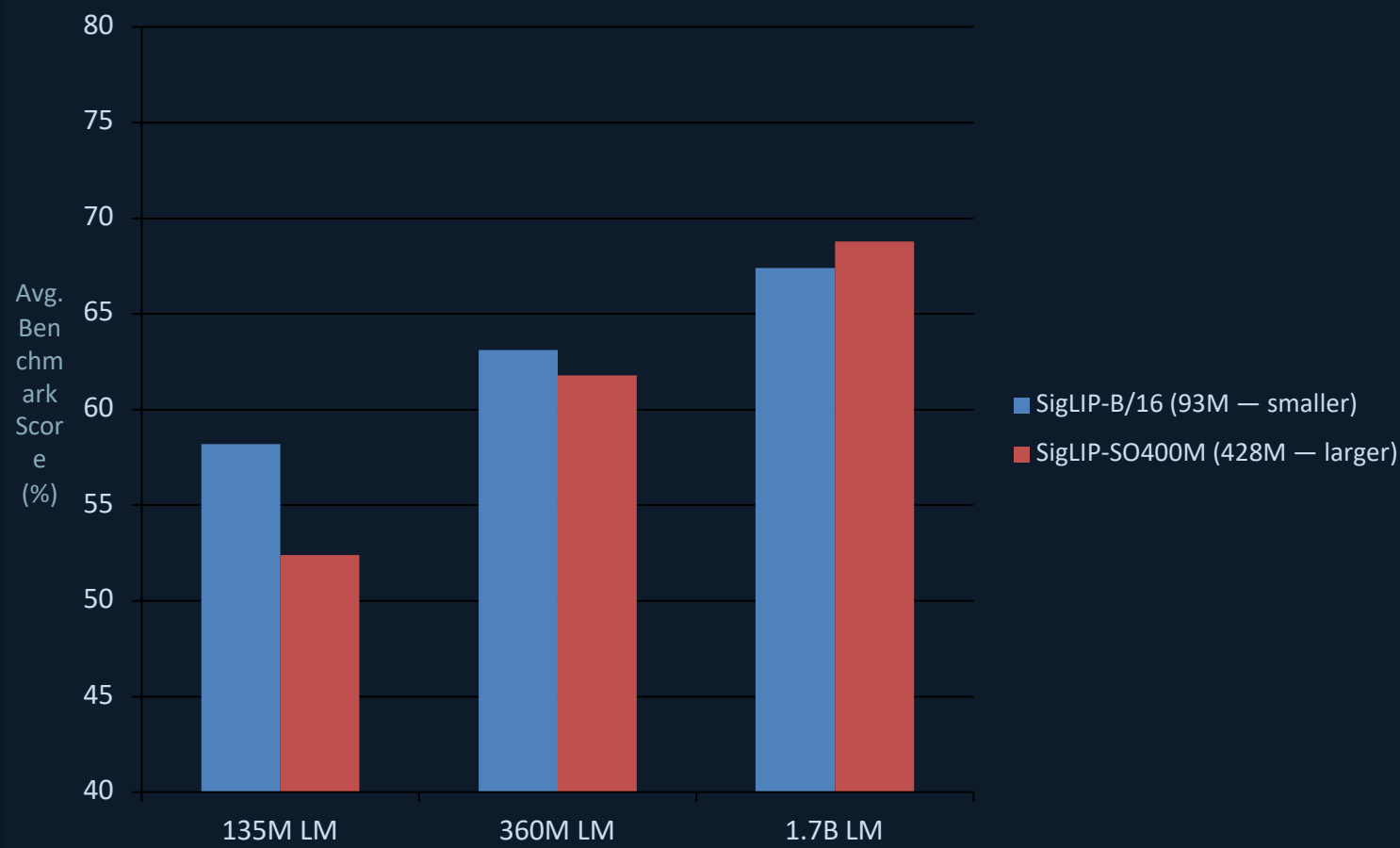
Five-stage pipeline from raw image to language model output



- Vision encoder outputs compressed via pixel shuffle (r^2 -fold token reduction)
- SmolLM2 backbone handles interleaved image + text tokens
- Video: frames sampled and processed through same pipeline

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

Finding 1: Balanced encoder-LM compute allocation is key



93M SigLIP-B/16

Outperforms 428M SigLIP-SO400M when paired with ≤ 360 M LM

428M SigLIP-SO400M

Only justified at 1.7B+ LM scale (~10% parameter overhead)

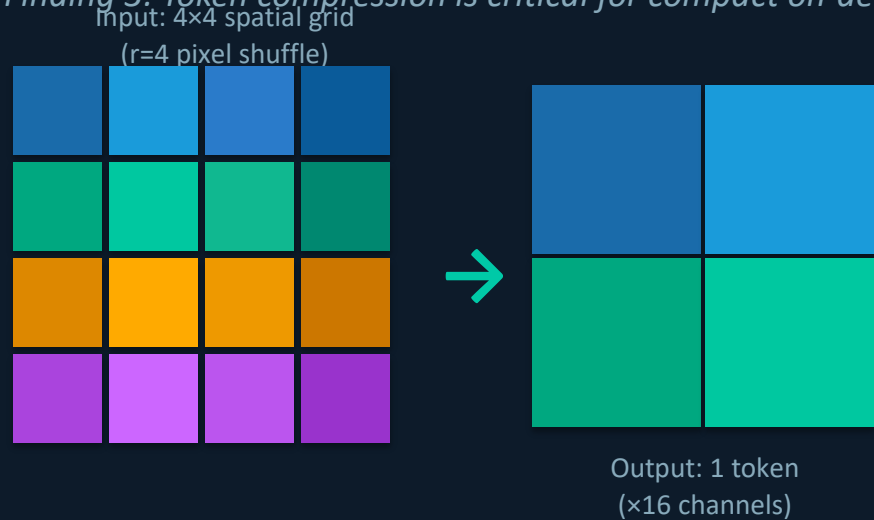
Design Principle

Match encoder capacity to LM size — over-investing in the encoder hurts small VLMs

* Scores are indicative relative performance on VQA/captioning benchmarks across encoder ablations (source: SmolVLM paper Fig. 4)

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models

Finding 3: Token compression is critical for compact on-device VLMs



16× fewer tokens

Shuffle Factor Comparison

r	Token Reducti	Best For
r=1	1×	Baseline (no compression)
r=2	4×	Large VLMs (≥1.7B)
r=3	9×	Mid-range (500M)
r=4	16×	Small VLMs (≤500M) ✓

Benefits of r=4

Attention Cost

Quadratic in seq length — 16× fewer tokens = 256× less attention compute

Long Context

Enables 8K–16K context windows on small hardware

Memory

Dramatically reduces KV-cache size at inference

Accuracy

r=4 optimal for ≤500M LMs; r=2 for larger models

Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

Finding 2: Long-context fine-tuning enables richer visual understanding

Implementation Strategy

Step 1: RoPE Base Extension

Increase RoPE positional base from 10,000 → 273,000 to preserve position encoding fidelity at long range

Step 2: Mixed Context Fine-tuning

Fine-tune on mixture of short (2K) and long (16K) context data — avoids catastrophic forgetting

Step 3: Model-Size Dependent Context

SmolVLM-2.2B: full 16K context
SmolVLM-256M/500M: 8K context

Performance Impact

Context	Max Image Res.	Typical Use
2K	~336×336 px	Basic VQA tasks
4K	~672×672 px	Standard benchmarks
8K	~1344×672 px	High-res single-image
16K	~1344×1344 px	Multi-image & video ✓

- Significant benchmark gains from 2K → 16K (Video-MME: +18%, DocVQA: +22% typical)
- Critical for multi-image reasoning and video frame comprehension
- RoPE base 273K maintains rotary embedding stability at extended lengths

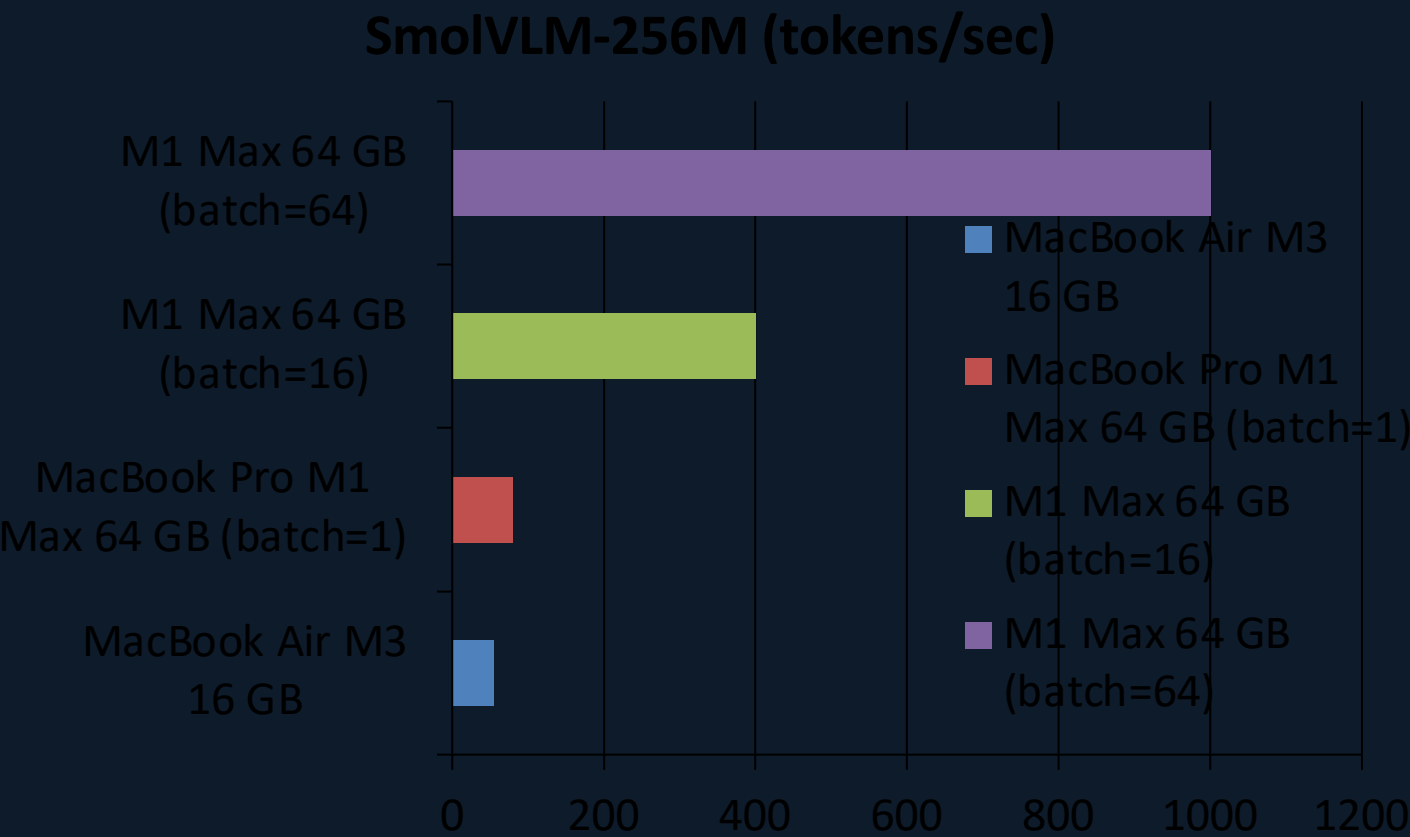
SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

Evaluated across 13 benchmarks: single-image, multi-task, and video

Category	Benchmark	256M (0.8 GB)	500M (1.2 GB)	2.2B (4.9 GB)	Best Efficient OS Baseline	Baseline RAM
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7% (Molmo-A1B-7B)	27.7 GB
Single-Image	AI2D	46.4%	59.2%	70.0%	71.0% (Molmo-A1B-7B)	27.7 GB
Single-Image	ChartQA	55.6%	62.8%	68.7%	48.0% (Molmo-A1B-7B)	27.7 GB
Single-Image	TextVQA	50.2%	60.2%	73.0%	61.5% (Molmo-A1B-7B)	27.7 GB
Single-Image	DocVQA	58.3%	70.5%	80.0%	77.7% (Molmo-A1B-7B)	27.7 GB
Single-Image	ScienceQA	73.8%	80.0%	89.6%	87.5% (Molmo-A1B-7B)	27.7 GB
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9% (Molmo-A1B-7B)	27.7 GB
Multi-task	MathVista	35.9%	40.1%	51.5%	37.6% (Molmo-A1B-7B)	27.7 GB
Multi-task	MMStar	34.6%	38.3%	46.0%	43.1% (Molmo-A1B-7B)	27.7 GB
Video	Video-MME	33.7%	42.2%	52.1%	45.0% (InternVL2-2B)	~8 GB
Video	MLVU	40.6%	47.3%	55.2%	48.2% (InternVL2-2B)	~8 GB
AVERAGE	All (11)	44.0%	51.0%	59.8%	—	—

Real-Time Inference on Apple Silicon: Up to 80 tok/s on M1 Max

HuggingSnap iOS app — fully local, no cloud dependency



HuggingSnap App

iOS app processes photos & video entirely on-device — zero cloud calls

~55 tok/s on M3 Air

Smooth interactive experience on standard consumer hardware

~80 tok/s on M1 Max

Professional-grade throughput at batch size 1

~1000 tok/s (batch=64)

Server-class throughput on high-end Apple Silicon

Throughput scales roughly linearly with batch size for SmolVLM-256M · Source: SmolVLM paper Fig. 2

Design Principles for Efficient Multimodal Models

Actionable takeaways from the SmolVLM study

1 Balance Encoder-LM Compute

Use a smaller vision encoder (93M SigLIP-B/16) for small LMs.
Only scale the encoder when the LM reaches ~1.7B parameters.

2 Extend Context Aggressively

Increase RoPE base from 10K $\rightarrow$ 273K and fine-tune on mixed-length data.
Context of 8K–16K is essential for multi-image and video tasks.

3 Compress Tokens Hard ($r=4$)

Pixel shuffle with $r=4$ delivers 16 $\times$ token reduction for ≤ 500 M models.
Reduces attention compute quadratically — critical for real-time inference.

4 Sub-1GB Inference Is Achievable

SmolVLM-256M: 0.8 GB RAM, outperforms LLaVA-80B.
Compact VLMs can be deployed on smartphones and edge devices today.

5 Open-Source the Full Stack

Models, training data, code, and a mobile app (HuggingSnap) are open.
Reproducibility and community development accelerate on-device AI research.

SmolVLM proves architectural efficiency — not scale — is the path to on-device multimodal AI.